RAMDEOBABA UNIVERSITY, NAGPUR

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

B.Tech. CSE (Artificial Intelligence and Data Science) (AIDS)

SEMESTER V

SI LAB

PROJECT REPORT

VIDEO GAME SALES ANALYSIS AND HIT PREDICTION

Submitted by

Himani Dalvi

Roll No.: 07

Enrollment No.: 24RBU/UG/CS/02/000203

Batch: B3-B1

Academic Year: 2026–27

Video Game Sales Analysis and Hit Prediction

SI Lab Project Report

1. PROJECT OBJECTIVE

The main purpose of this project is to study video game sales data using the statistical methods we learned in the SI Lab, and then build machine learning models that can predict how well a game will sell. The project tries to answer two simple questions: will a game become a hit (1 million or more copies sold worldwide), and how many copies will it sell worldwide?

The expected output is a set of trained and compared models, and a small web app where a user enters a game's North America sales and gets both predictions. A game company or a student can use it to get a quick idea of how a game may perform worldwide from its North America sales.

Objectives of the project:

- Collect data using web scraping and load the Kaggle Video Game Sales dataset.
- Explore the data using descriptive statistics and charts.
- Clean the data by handling missing values and checking duplicates.
- Apply sampling, probability distributions, CLT, confidence intervals, hypothesis tests (t-test, Z-test, ANOVA, chi-square) and correlation.
- Select the input feature using correlation analysis.
- Train 4 classification models to predict Hit or Not Hit, and 5 regression models to predict Global Sales.
- Evaluate and compare the models using accuracy, precision, recall, F1-score and confusion matrix (classification), and MAE, MSE, RMSE and R2 (regression).
- Select the best models and save them using joblib.
- Develop a Streamlit GUI that accepts input and displays predictions.

2. INTRODUCTION

The video game industry is one of the biggest entertainment industries in the world. Thousands of games are released every year on platforms like PlayStation, Xbox, Nintendo and PC, but only a few of them become big hits. Most games sell only a small number of copies.

Sales data can tell us a lot about this market: which genres sell the most, which platforms are popular and which regions buy the most games. In this project we use the Video Game Sales dataset, which has 16,598 games released between 1980 and 2020 along with their sales in North America, Europe, Japan and other regions.

The motivation for this project is to apply all the SI Lab practicals (PR1 to PR7) on one real dataset and then go one step further by building a prediction tool. North America is the biggest market in the data (about 49% of all sales), so a game's North America sales can give a good early idea of how it will do in the rest of the world.

2.1 Problem Statement

Given the North America sales of a video game, predict (1) whether the game will become a hit, which means it sells 1 million or more copies worldwide, and (2) how many copies it will sell worldwide. Along with this, use statistical tests to find out how game sales depend on genre, platform and region.

2.2 Scope of the Project

The project includes:

- Web scraping of the best-selling video games list from Wikipedia.
- Data cleaning and exploratory data analysis with charts.
- Sampling methods, probability distributions, CLT, point estimation, bootstrap and confidence intervals.
- Hypothesis testing: t-tests, Z-test, one-way and two-way ANOVA, chi-square tests and correlation.
- 4 classification models and 5 regression models, with a full comparison.
- A Streamlit web app for predictions.

Limitations:

- The models use only one input (NA Sales), so they can miss games that are popular mainly in Japan or Europe.
- The dataset has data only up to about 2016, so the last few years are incomplete.
- The Streamlit app runs on the local computer (localhost). To share it with others, it has to be deployed online, for example on Streamlit Community Cloud.

3. PROJECT METHODOLOGY

The project was done in the following stages:

Data Collection → Loading and Exploration → Statistical Analysis → Data Preprocessing → Feature Selection → Model Training → Evaluation and Comparison → Best Model Selection → Streamlit GUI

3.1 Dataset Loading and Exploration

- **Dataset name:** Video Game Sales (vgsales.csv).
- **Source:** Kaggle, https://www.kaggle.com/datasets/dandanjia/vgsales-csv. The data was originally collected from vgchartz.com.
- **Size:** 16,598 rows and 11 columns. It has 12 genres, 31 platforms and 576 publishers.
- **Target variable:** for classification we created a new column Hit (1 if Global_Sales is 1 million or more, otherwise 0). For regression the target is Global_Sales.
- **Missing values:** Year has 271 missing values and Publisher has 58. There are no duplicate records.

Table 1: Columns of the dataset

Column	Data Type	Description
Rank	int64	Rank of the game by global sales
Name	object	Name of the game
Platform	object	Platform on which the game was released (Wii, PS2, DS, X360 etc.)
Year	float64	Year of release
Genre	object	Genre of the game (Action, Sports, Shooter etc.)
Publisher	object	Company that published the game
NA_Sales	float64	Sales in North America (in millions)
EU_Sales	float64	Sales in Europe (in millions)
JP_Sales	float64	Sales in Japan (in millions)
Other_Sales	float64	Sales in the rest of the world (in millions)
Global_Sales	float64	Total worldwide sales (in millions)

```
df = pd.read_csv("vgsales.csv")
print(df.shape)
df.head()
```

```
(16598, 11)
```

	Rank	Name	Platform	Year	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
0	1	Wii Sports	Wii	2006.0	Sports	Nintendo	41.49	29.02	3.77	8.46	82.74
1	2	Super Mario Bros.	NES	1985.0	Platform	Nintendo	29.08	3.58	6.81	0.77	40.24
2	3	Mario Kart Wii	Wii	2008.0	Racing	Nintendo	15.85	12.88	3.79	3.31	35.82
3	4	Wii Sports Resort	Wii	2009.0	Sports	Nintendo	15.75	11.01	3.28	2.96	33.00
4	5	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	Nintendo	11.27	8.89	10.22	1.00	31.37

Fig. 1: First five rows of the dataset

The dataset is sorted by global sales. Wii Sports is the best-selling game with 82.74 million copies.

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 16598 entries, 0 to 16597
Data columns (total 11 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Rank          16598 non-null  int64
 1   Name          16598 non-null  str
 2   Platform      16598 non-null  str
 3   Year          16327 non-null  float64
 4   Genre         16598 non-null  str
 5   Publisher     16540 non-null  str
 6   NA_Sales      16598 non-null  float64
 7   EU_Sales      16598 non-null  float64
 8   JP_Sales      16598 non-null  float64
 9   Other_Sales   16598 non-null  float64
 10  Global_Sales  16598 non-null  float64
dtypes: float64(6), int64(1), str(4)
memory usage: 1.4 MB
```

Fig. 2: Dataset information

The dataset has 7 numeric and 4 text columns. Year and Publisher have fewer non-null values than the other columns, which means they have missing values.

```
print("Missing values:")
print(df.isnull().sum())
```

```
Missing values:
Rank              0
Name              0
Platform          0
Year            271
Genre             0
Publisher        58
NA_Sales          0
EU_Sales          0
JP_Sales          0
Other_Sales       0
Global_Sales      0
dtype: int64
```

Fig. 3: Missing values in each column

Only Year (271) and Publisher (58) have missing values. All sales columns are complete.

Descriptive statistics of Global Sales:

Mean = 0.54 million, Median = 0.17 million, Mode = 0.02 million, Standard Deviation = 1.57, Q1 = 0.06, Q3 = 0.48, IQR = 0.42, Skewness = 17.3. The mean is much higher than the median, so the data is strongly right skewed: most games sell very little and a few hit games sell a lot.

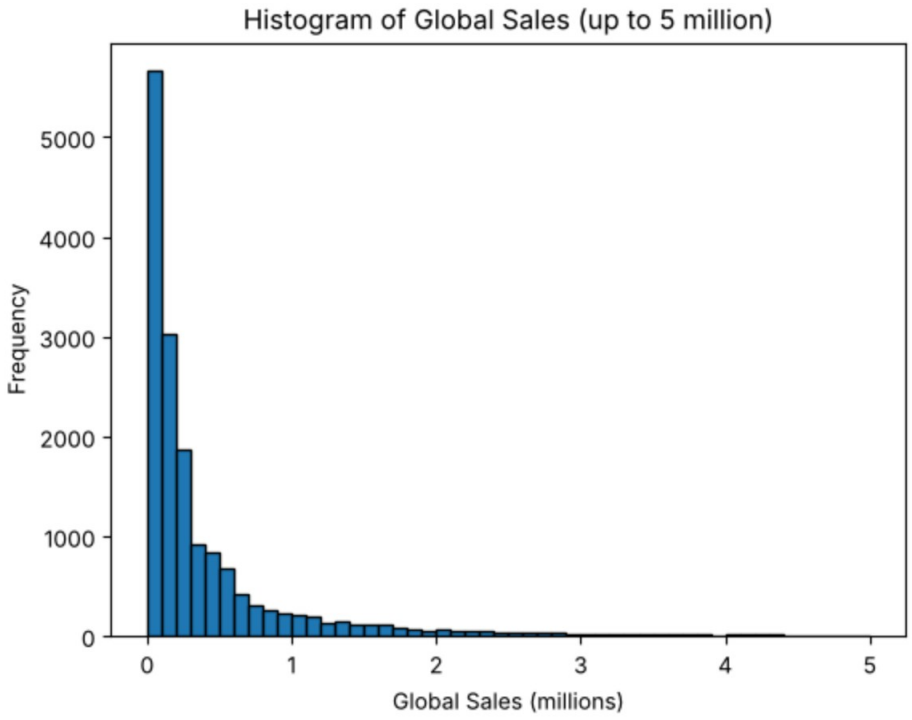


Fig. 4: Histogram of Global Sales (up to 5 million)

Most games sold less than 1 million copies. The long tail on the right shows the few big hits.

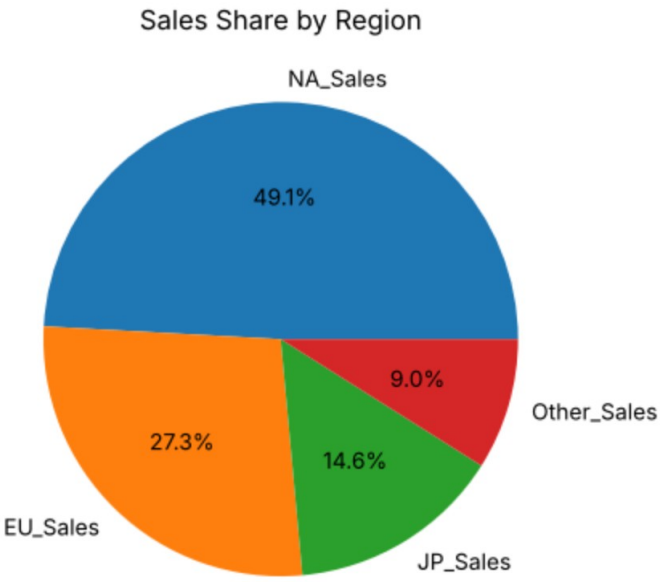
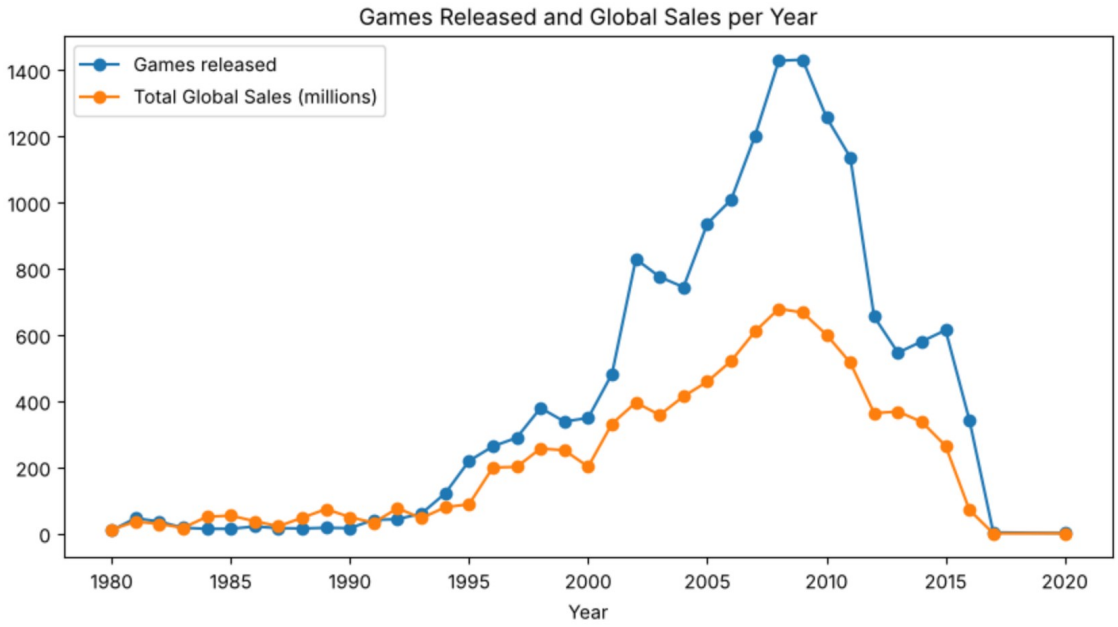


Fig. 5: Sales share by region

North America has the biggest share of sales (49.1%), followed by Europe (27.3%), Japan (14.6%) and other regions (9.0%).




```
Year with most games: 2009
Year with highest sales: 2008
```

Fig. 6: Games released and global sales per year

Game releases and sales grew quickly after 1995 and peaked in 2008 and 2009. The data stops around 2016, so the last few years look very low.

```
Hit
0    14233
1     2058
Name: count, dtype: int64
```

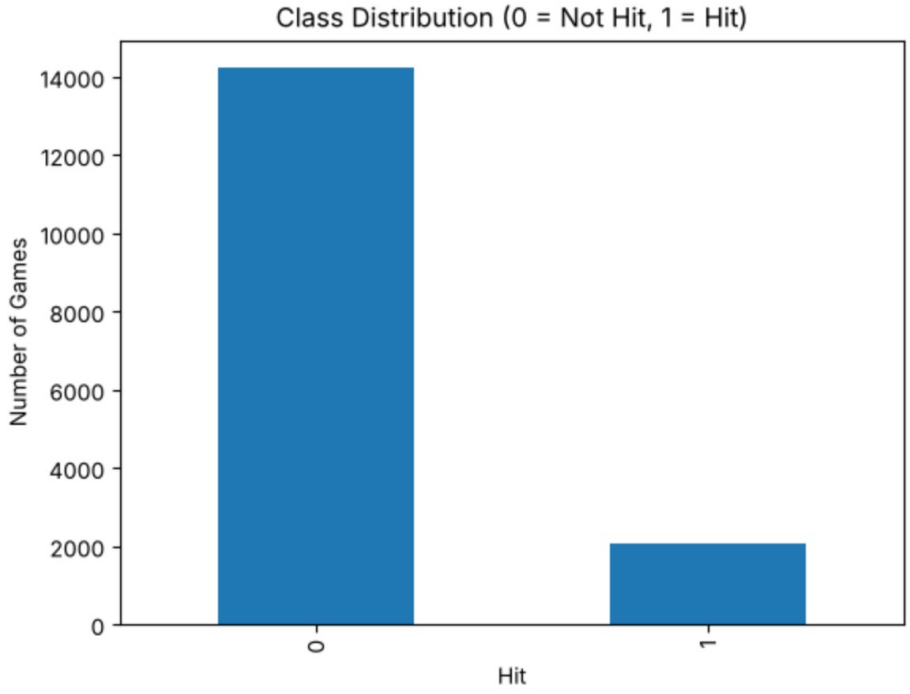


Fig. 7: Class distribution of the target (Hit)

Only 2,058 out of 16,291 games (12.6%) are hits. The classes are imbalanced, which affects how we choose the best model.

Statistical analysis (SI Lab practicals):

Before building the models, all statistical practicals were applied on the same data. The main results are given below. The full code and output for each test is in the Colab notebook.

Table 2: Summary of statistical analysis

Method / Test	What we checked	Result and conclusion
Sampling (6 methods)	Simple random, with replacement, fraction, systematic, stratified and cluster sampling	Simple random sample mean = 0.538 vs population mean = 0.541 (only 0.6% error)
Probability distributions	Normal, binomial, Poisson, uniform, PMF and CDF	P(no hit in 5 random games) = 0.51. 2008 and 2009 are the most common release years
CLT and point estimation	Distribution of 1000 sample means (n = 30)	Sample means look normal. Observed SE = 0.278, theoretical SE = 0.286
Bootstrap and estimators	Sample mean vs sample median	Median has much lower variance (0.004 vs 0.071), so it is more stable for skewed data
Confidence interval	95% CI of mean Global Sales (t and Z)	0.517 to 0.565 million
One-sample t-test and Z-test	Is mean Global Sales = 0.5 million?	t = Z = 3.33, p = 0.0009. Reject H0, the mean is different from 0.5
Two-sample t-test	Action vs Sports mean sales	p = 0.386. No significant difference
Paired t-test	NA sales vs EU sales of the same games	p close to 0. NA sales are significantly higher
One-way ANOVA	Do genres have different mean sales?	F = 18.18, p close to 0. Yes, Platform genre is highest (0.95), Adventure lowest (0.18)
Two-way ANOVA	Effect of genre, platform and their interaction	All three are significant ($p < 0.05$)
Chi-square (independence)	Are genre and platform related?	Chi-square = 591.5, DOF = 16, p close to 0. Yes, they are related
Chi-square (goodness of fit)	Are games equally spread over top 5 genres?	p close to 0. No, Action has far more games
Correlation	Pearson and Spearman with Global Sales	NA r = 0.94, EU r = 0.90, JP r = 0.61. All significant

3.2 Data Preprocessing

- **Missing values:** rows with missing Year or Publisher were removed using dropna(). 307 rows were removed (22 rows had both missing), and 16,291 rows (98.2% of the data) were kept.
- **Data type:** Year was converted from float to integer.
- **Duplicates:** checked with drop_duplicates(). No duplicate records were found.

- **Encoding:** not needed, because the models use only the numeric NA_Sales column. Genre and Platform were used only in the statistical tests, where they are grouped directly.
- **Target creation:** a new column Hit was created (1 if Global_Sales ≥ 1 million, otherwise 0).
- **Input and target:** X = NA_Sales. y = Hit for classification and Global_Sales for regression.
- **Train-test split:** 80% training (13,032 games) and 20% testing (3,259 games) with random_state = 42. For classification stratify = y was used, so both sets keep the same 12.6% share of hits.
- **Feature scaling:** StandardScaler was fitted on the training data and then applied to the test data. Scaling is important for Logistic Regression and KNN.

The preprocessing outputs are shown in Fig. 9 and Fig. 10 in Section 6.1.

3.3 Feature Extraction / Selection

No separate feature extraction technique was needed. The input feature was selected using correlation analysis. NA_Sales has the strongest correlation with Global_Sales (Pearson r = 0.94), followed by EU_Sales (0.90) and JP_Sales (0.61). NA_Sales was selected because it has the strongest relation, it is the biggest market, and a single input keeps the GUI simple.

Some columns were not used on purpose. Global_Sales is the sum of all regional sales, so using all regions together as input would make the prediction trivial. Rank was also not used because it is calculated from Global_Sales. Name has no predictive value.

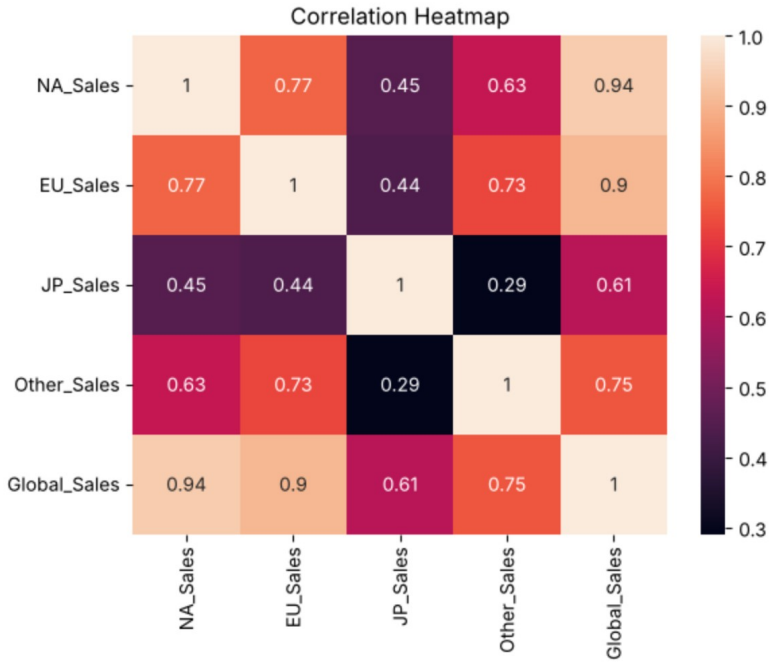


Fig. 8: Correlation heatmap of sales columns

NA_Sales and EU_Sales have the strongest relation with Global_Sales, so NA_Sales was chosen as the input feature.

3.4 Training Multiple Classification Models

Four classification models were trained to predict whether a game will be a Hit (1) or Not Hit (0).

Table 3: Classification models used

Sr. No.	Model Name	Purpose / Notes
1	Logistic Regression	A simple linear model that gives the probability of a game being a hit. Used as the baseline model.
2	K-Nearest Neighbors (k = 5)	Looks at the 5 most similar games in the training data and takes the majority class.
3	Decision Tree (max_depth = 5)	Learns simple if-else rules on NA Sales (for example, if NA Sales is above a certain value then Hit). Easy to understand and explain.
4	Random Forest (100 trees, max_depth = 5)	Combines 100 decision trees and takes a vote. This usually gives more stable results than a single tree.

Five regression models were also trained (PR7) to predict the exact Global Sales value: Linear Regression, Ridge Regression (alpha = 1.0), Lasso Regression (alpha = 0.1), Decision Tree Regressor (max_depth = 5) and Random Forest Regressor (100 trees, max_depth = 5).

3.5 Performance Evaluation and Model Comparison

The classification models were evaluated on the 3,259 test games using these metrics:

- **Accuracy:** percentage of all games that were predicted correctly.
- **Precision:** out of the games predicted as hits, how many were really hits.
- **Recall:** out of the real hits, how many the model found.
- **F1-Score:** a balance of precision and recall. It is useful when the classes are imbalanced.
- **Confusion matrix:** shows correct and wrong predictions for each class.

Table 4: Classification results (test data)

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.9583	0.9157	0.7379	0.8172
KNN	0.9494	0.8402	0.7403	0.7871
Decision Tree	**0.9580**	**0.8873**	**0.7646**	**0.8214**
Random Forest	0.9573	0.8802	0.7670	0.8197

The regression models were evaluated using MAE (average error), MSE, RMSE (error in millions of copies) and R2 (how much of the variation in sales the model explains, 1 is perfect).

Table 5: Regression results (test data)

Model	MAE	MSE	RMSE	R2
Linear Regression	**0.2098**	**0.3384**	**0.5818**	**0.9209**
Ridge Regression	0.2098	0.3385	0.5818	0.9209
Lasso Regression	0.2210	0.4067	0.6378	0.9049
Decision Tree	0.2228	0.8850	0.9408	0.7931
Random Forest	0.2165	0.9711	0.9855	0.7730

All values in Table 4 and Table 5 are taken from the output of our own Colab notebook (Fig. 11 and Fig. 12).

3.6 Best Model Selection

Classification: Decision Tree

The Decision Tree was selected as the best classification model because it has the highest F1-Score (0.8214). We used F1-Score instead of accuracy because only 12.6% of the games are hits. A model that always says “Not Hit” would already get about 87% accuracy, so accuracy alone can be misleading.

Logistic Regression has almost the same accuracy (0.9583) and a higher precision, but it finds fewer real hits (recall 0.7379). The Decision Tree finds more real hits (recall 0.7646) while keeping precision high (0.8873). Random Forest is very close, but it uses 100 trees, while a single Decision Tree is simpler and easier to explain.

Regression: Linear Regression

Linear Regression was selected as the best regression model because it has the highest R2 (0.9209) and the lowest RMSE (0.5818). It also gives a simple equation: Global Sales = 0.53 + 1.32 × NA Sales (scaled).

Both best models and their scalers were saved using joblib as best_classifier.pkl, scaler_classifier.pkl, best_model.pkl and scaler.pkl. These files are loaded again by the GUI.

4. PROJECT IMPLEMENTATION

4.1 Tools and Technologies

Tool / Technology	Version	Purpose
Python	3.13.16	Programming language
Google Colab	Free (CPU runtime)	Development environment
Pandas / NumPy	2.2.3 / 2.1.3	Data handling
Scikit-learn	1.6.1	Preprocessing and models
Matplotlib / Seaborn	3.10.0 / 0.13.2	Visualization
SciPy / Statsmodels	1.16.3 / Colab default	Statistical tests (t-test, ANOVA, chi-square)

Tool / Technology	Version	Purpose
Requests / BeautifulSoup	Colab default	Web scraping
Joblib	Colab default	Saving and loading models
GUI framework	Streamlit 1.66.1	Web app user interface

4.2 Module-wise Implementation

- **Data collection module:** requests and BeautifulSoup are used to scrape the best-selling games table from Wikipedia. The data is cleaned and saved as best_selling_games.csv.
- **Dataset loading and exploration module:** read_csv(), head(), info(), isnull(), describe(), value_counts() and charts (histogram, boxplot, bar, pie, scatter and line chart).
- **Statistical analysis module:** sampling methods, probability distributions, CLT, bootstrap, confidence intervals, t-tests, Z-test, ANOVA, chi-square tests and correlation using scipy.stats and statsmodels.
- **Data preprocessing module:** dropna(), astype(), drop_duplicates(), creation of the Hit column, train_test_split() and StandardScaler.
- **Feature selection module:** pearsonr(), spearmanr() and a correlation heatmap to choose NA_Sales as the input.
- **Model training and evaluation module:** 4 classifiers and 5 regressors are trained in a loop. Accuracy, precision, recall, F1-score, confusion matrix, MAE, MSE, RMSE and R2 are calculated and stored in a comparison table.
- **Best model saving/loading module:** the best model is picked from the sorted table and saved with joblib.dump(). It is loaded again with joblib.load().
- **GUI input and prediction module:** a predict_game() function takes NA Sales, scales it and returns the predicted global sales, the chance of Hit or Not Hit, a short summary and a small chart table. Streamlit shows this as a web app (app.py).

4.3 GUI Implementation

The GUI is built with Streamlit in a separate file, app.py, using a wide two-column layout (st.columns). The left column has a slider (st.slider) to choose NA Sales from 0 to 10 million, Clear and Predict buttons, and a bar chart (st.bar_chart) of NA Sales vs predicted Global Sales. The right column shows the “Predicted Global Sales (millions)” as a metric, a green “HIT game” or red “Not a hit” box with progress bars for the chance of each, and a “Summary” box. A line at the bottom shows the models used and their scores.

When the user clicks Predict, the predict_game() function puts the value in a DataFrame and scales it with the saved scalers. The Linear Regression model predicts the global sales. The Decision Tree model gives the probability of Hit and Not Hit using predict_proba(), which is shown as progress bars. The Summary box compares the predicted sales with all games in the dataset and shows what percentage of games it beats, and the bar chart shows how NA Sales grow into worldwide sales. The Clear button resets everything, and moving the slider hides the old result until Predict is clicked again. The models and the dataset are loaded only once using

st.cache_resource and st.cache_data. The app is started with the command streamlit run app.py, which opens it in the browser at http://localhost:8501.

For example, for NA Sales = 1.5 million the app predicts 2.69 million global sales and “HIT game” with 100% probability, which is more than 96.6% of all games. For NA Sales = 0.3 million it predicts 0.6 million global sales and “Not a hit” with 94% probability. The screenshots are shown in Fig. 14, Fig. 15 and Fig. 16 in Section 6.1.

5. COMPLETE CODE

The complete runnable code is in the Colab notebook B3_B1_07_SIPROJECT.ipynb, which covers all SI Lab practicals from PR1 to PR7, and in the GitHub repository (Section 8). The main machine learning and GUI code is given below. It is the same code that produced the results and screenshots in this report.

```
# ---------- 1. Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

# ---------- 2. Load the dataset
df = pd.read_csv("vgsales.csv")
print(df.shape)
df.head()

# ---------- 3. Preprocessing: remove missing values and duplicates
df = df.dropna()

df["Year"] = df["Year"].astype(int)
df = df.drop_duplicates()

print("Clean data shape:", df.shape)
print("Missing values left:", df.isnull().sum().sum())

# ---------- 4. PART A: Regression (predict Global Sales from NA Sales)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

X = df[["NA_Sales"]]      #Independent variable
Y = df["Global_Sales"]    #Dependent variable

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)
print("Training Sample:", len(X_train))
print("Testing Sample:", len(X_test))

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

#Train all 5 models and compare
models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Lasso Regression": Lasso(alpha=0.1),
    "Decision Tree": DecisionTreeRegressor(max_depth=5, random_state=42),
    "Random Forest": RandomForestRegressor(n_estimators=100, max_depth=5, random_state=42)
}

results = []
for name, m in models.items():
    m.fit(X_train, Y_train)
    pred = m.predict(X_test)
    mae = mean_absolute_error(Y_test, pred)
```

```
    mse = mean_squared_error(Y_test, pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(Y_test, pred)
    results.append([name, mae, mse, rmse, r2])

results_df = pd.DataFrame(results, columns=["Model", "MAE", "MSE", "RMSE", "R2"])
results_df = results_df.sort_values(by="R2", ascending=False)
results_df

#Best model
best_model_name = results_df.iloc[0]["Model"]
best_model = models[best_model_name]
print("Best Model:", best_model_name)

#Save and load the model
import joblib
joblib.dump(best_model, "best_model.pkl")
joblib.dump(scaler, "scaler.pkl")

model = joblib.load("best_model.pkl")
scaler = joblib.load("scaler.pkl")

# ---------- 5. PART B: Classification (Hit or Not Hit)
#Target: Hit = 1 if Global Sales >= 1 million, else 0
df["Hit"] = (df["Global_Sales"] >= 1).astype(int)
print(df["Hit"].value_counts())

df["Hit"].value_counts().plot(kind="bar")
plt.title("Class Distribution (0 = Not Hit, 1 = Hit)")
plt.xlabel("Hit")
plt.ylabel("Number of Games")
plt.show()

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

Xc = df[["NA_Sales"]]     #Input feature
yc = df["Hit"]            #Target (0 or 1)

Xc_train, Xc_test, yc_train, yc_test = train_test_split(Xc, yc, test_size=0.2,
random_state=42, stratify=yc)
print("Training Sample:", len(Xc_train))
print("Testing Sample:", len(Xc_test))

scaler_c = StandardScaler()
Xc_train = scaler_c.fit_transform(Xc_train)
Xc_test = scaler_c.transform(Xc_test)

#Train 4 classification models and compare
classifiers = {
    "Logistic Regression": LogisticRegression(),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(max_depth=5, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
}

c_results = []
for name, c in classifiers.items():
    c.fit(Xc_train, yc_train)
    pred = c.predict(Xc_test)
    c_results.append([name,
                      accuracy_score(yc_test, pred),
                      precision_score(yc_test, pred),
                      recall_score(yc_test, pred),
                      f1_score(yc_test, pred)])

c_results_df = pd.DataFrame(c_results, columns=["Model", "Accuracy", "Precision", "Recall",
"F1-Score"])
c_results_df = c_results_df.sort_values(by="F1-Score", ascending=False)
c_results_df

#Best classification model (chosen by F1-Score)
best_clf_name = c_results_df.iloc[0]["Model"]
```

```
best_clf = classifiers[best_clf_name]
print("Best Classification Model:", best_clf_name)

cm = confusion_matrix(yc_test, best_clf.predict(Xc_test))
print(cm)

ConfusionMatrixDisplay(cm, display_labels=["Not Hit", "Hit"]).plot()
plt.title("Confusion Matrix - " + best_clf_name)
plt.show()

#Save and load the classification model
joblib.dump(best_clf, "best_classifier.pkl")
joblib.dump(scaler_c, "scaler_classifier.pkl")

clf = joblib.load("best_classifier.pkl")
scaler_c = joblib.load("scaler_classifier.pkl")

# ---------- 6. Streamlit GUI (app.py)
# Video Game Sales and Hit Predictor (Streamlit app)
# Run:  streamlit run app.py
# Keep vgsales.csv and the 4 .pkl files in the same folder as this file.

import joblib
import pandas as pd
import streamlit as st

st.set_page_config(page_title="Video Game Sales and Hit Predictor", page_icon="🎮",
layout="wide")


# load the saved best models, scalers and the dataset (only once)
@st.cache_resource
def load_models():
    model = joblib.load("best_model.pkl")              # Linear Regression (Global Sales)
    scaler = joblib.load("scaler.pkl")
    clf = joblib.load("best_classifier.pkl")           # Decision Tree (Hit / Not Hit)
    scaler_c = joblib.load("scaler_classifier.pkl")
    return model, scaler, clf, scaler_c


@st.cache_data
def load_data():
    return pd.read_csv("vgsales.csv").dropna()        # used for the summary


model, scaler, clf, scaler_c = load_models()
df = load_data()


def predict_game(na_sales):
    x = pd.DataFrame({"NA_Sales": [na_sales]})
    sales = round(float(model.predict(scaler.transform(x))[0]), 2)  # regression model
    prob = clf.predict_proba(scaler_c.transform(x))[0]  # classification model
    hit = {"HIT game": float(prob[1]), "Not a hit": float(prob[0])}
    beaten = round((df["Global_Sales"] < sales).mean() * 100, 1)  # compare with dataset
    summary = (f"A game that sells {na_sales} million copies in North America is expected to
sell "
               f"about {sales} million copies worldwide. That is more than {beaten}% of all
games in the dataset.")
    chart = pd.DataFrame({"Region": ["North America", "Worldwide"],
                          "Sales (millions)": [na_sales, sales]})
    return sales, hit, summary, chart


# remember the slider value and the last result
if "na" not in st.session_state:
    st.session_state.na = 1.0
    st.session_state.result = None


def on_predict():
    st.session_state.result = predict_game(st.session_state.na)


def on_clear():
    st.session_state.na = 1.0
    st.session_state.result = None
```

```
def on_slide():
    st.session_state.result = None     # old result hides when the slider moves


# ---------- page layout
st.title("🎮 Video Game Sales and Hit Predictor")

left, right = st.columns(2, gap="large")

with left:
    st.slider("NA Sales (millions)", 0.0, 10.0, step=0.1, key="na", on_change=on_slide)
    b1, b2 = st.columns(2)
    b1.button("Clear", on_click=on_clear, width="stretch")
    b2.button("Predict", on_click=on_predict, type="primary", width="stretch")
    if st.session_state.result:
        st.subheader("NA Sales vs Predicted Global Sales")
        st.bar_chart(st.session_state.result[3], x="Region", y="Sales (millions)")

with right:
    if st.session_state.result:
        sales, hit, summary, chart = st.session_state.result
        st.metric("Predicted Global Sales (millions)", sales)
        if hit["HIT game"] >= 0.5:
            st.success("HIT game")
        else:
            st.error("Not a hit")
        st.progress(hit["HIT game"], text=f"HIT game: {hit['HIT game'] * 100:.0f}%")
        st.progress(hit["Not a hit"], text=f"Not a hit: {hit['Not a hit'] * 100:.0f}%")
        st.info(summary)
    else:
        st.info("Set NA Sales on the slider and click Predict.")

st.caption("Models used: Linear Regression for global sales (R2 = 0.92) and Decision Tree for
hit prediction (accuracy 95.8%).")
```

6. RESULTS AND DISCUSSION

6.1 Required Output Screenshots

```
print("Clean data shape:", df.shape)
print("Missing values left:", df.isnull().sum().sum())
```

```
Clean data shape: (16291, 11)
Missing values left: 0
```

Fig. 9: Data preprocessing output

After removing missing values, 16,291 rows are left and no missing values remain.

```
Xc = df[["NA_Sales"]]     #Input feature
yc = df["Hit"]            #Target (0 or 1)

Xc_train, Xc_test, yc_train, yc_test = train_test_split(Xc, yc, test_size=0.2, random_state=42, stratify=yc)
print("Training Sample:", len(Xc_train))
print("Testing Sample:", len(Xc_test))

scaler_c = StandardScaler()
Xc_train = scaler_c.fit_transform(Xc_train)
Xc_test = scaler_c.transform(Xc_test)
```

```
Training Sample: 13032
Testing Sample: 3259
```

Fig. 10: Train-test split and feature scaling

13,032 games are used for training and 3,259 for testing. NA_Sales is scaled using StandardScaler.

```python
#Train 4 classification models and compare
classifiers = {
    "Logistic Regression": LogisticRegression(),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(max_depth=5, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
}

c_results = []
for name, c in classifiers.items():
    c.fit(Xc_train, yc_train)
    pred = c.predict(Xc_test)
    c_results.append([name,
                      accuracy_score(yc_test, pred),
                      precision_score(yc_test, pred),
                      recall_score(yc_test, pred),
                      f1_score(yc_test, pred)])

c_results_df = pd.DataFrame(c_results, columns=["Model", "Accuracy", "Precision", "Recall", "F1-Score"])
c_results_df = c_results_df.sort_values(by="F1-Score", ascending=False)
c_results_df
```

	Model	Accuracy	Precision	Recall	F1-Score
2	Decision Tree	0.957963	0.887324	0.764563	0.821382
3	Random Forest	0.957349	0.880223	0.766990	0.819715
0	Logistic Regression	0.958269	0.915663	0.737864	0.817204
1	KNN	0.949371	0.840220	0.740291	0.787097

Fig. 11: Classification model comparison table

All four models have accuracy between 95% and 96%. The Decision Tree has the highest F1-Score (0.8214).

```python
#Train all 5 models and compare
models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Lasso Regression": Lasso(alpha=0.1),
    "Decision Tree": DecisionTreeRegressor(max_depth=5, random_state=42),
    "Random Forest": RandomForestRegressor(n_estimators=100, max_depth=5, random_state=42)
}

results = []
for name, m in models.items():
    m.fit(X_train, Y_train)
    pred = m.predict(X_test)
    mae = mean_absolute_error(Y_test, pred)
    mse = mean_squared_error(Y_test, pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(Y_test, pred)
    results.append([name, mae, mse, rmse, r2])

results_df = pd.DataFrame(results, columns=["Model", "MAE", "MSE", "RMSE", "R2"])
results_df = results_df.sort_values(by="R2", ascending=False)
results_df
```

	Model	MAE	MSE	RMSE	R2
0	Linear Regression	0.209798	0.338439	0.581755	0.920888
1	Ridge Regression	0.209805	0.338489	0.581798	0.920876
2	Lasso Regression	0.220954	0.406735	0.637758	0.904923
3	Decision Tree	0.222789	0.885035	0.940763	0.793118
4	Random Forest	0.216458	0.971135	0.985462	0.772992

Fig. 12: Regression model performance results

Linear Regression has the highest R2 (0.9209) and the lowest error. Ridge is almost the same.

```python
#Best classification model (chosen by F1-Score)
best_clf_name = c_results_df.iloc[0]["Model"]
best_clf = classifiers[best_clf_name]
print("Best Classification Model:", best_clf_name)

cm = confusion_matrix(yc_test, best_clf.predict(Xc_test))
print(cm)

ConfusionMatrixDisplay(cm, display_labels=["Not Hit", "Hit"]).plot()
plt.title("Confusion Matrix - " + best_clf_name)
plt.show()
```

```
Best Classification Model: Decision Tree
[[2807   40]
 [  97  315]]
```

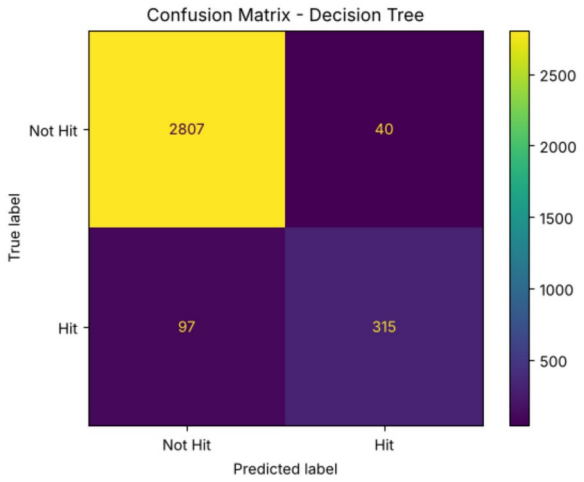


Fig. 13: Best model selection and confusion matrix

The Decision Tree is selected. It predicts 2,807 non-hit games and 315 hit games correctly. It wrongly calls 40 games hits and misses 97 real hits.

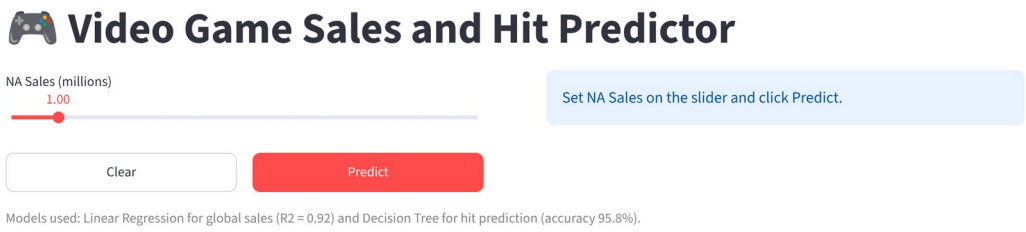


Fig. 14: Streamlit GUI interface

The user sets NA Sales on the slider and clicks Predict. Until then, the app shows a short message asking the user to do this.

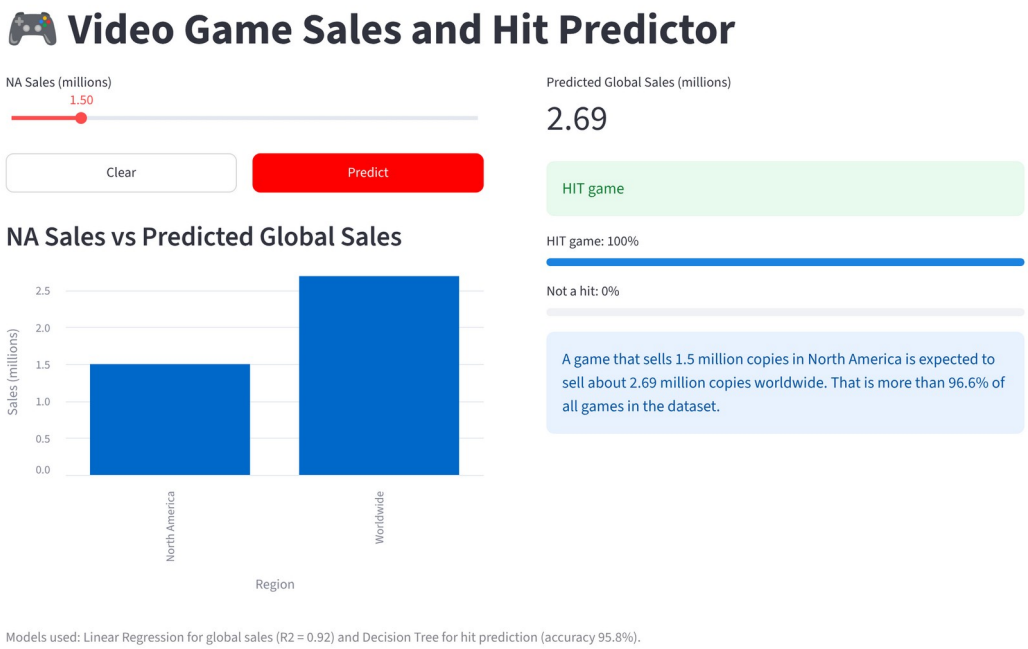


Fig. 15: GUI prediction result (Hit)

For NA Sales = 1.5 million, the app predicts 2.69 million global sales and HIT game (100%). The summary says this is more than 96.6% of all games, and the chart compares 1.5 with 2.69.



Fig. 16: GUI prediction result (Not Hit)

For NA Sales = 0.3 million, the app predicts 0.6 million global sales and Not a hit (94%). The summary says this is more than 79.5% of all games, and the chart compares 0.3 with 0.6.

6.2 Result Analysis

- **Data:** video game sales are highly right skewed (skewness 17.3). Most games sell less than 0.5 million copies, and a few hits like Wii Sports sell tens of millions. North America is the biggest market with 49.1% of all sales.
- **Statistics:** the tests show that genre and platform both have a significant effect on sales, and that they are related to each other. NA sales are significantly higher than EU sales and have the strongest correlation with global sales (r = 0.94). The CLT works well even on this skewed data.
- **Classification:** all four models reach 95% to 96% accuracy, but accuracy hides the class imbalance. F1-Score shows the real difference: Decision Tree (0.82), Random Forest (0.82), Logistic Regression (0.82) and KNN (0.79). KNN is the weakest because it makes more false hit predictions (lowest precision, 0.84).
- **Errors:** the Decision Tree missed 97 real hits. When we checked these games, 84% of them sold more in Japan and Europe than in North America. Since the model only looks at NA Sales, it cannot catch these games. This is the main weakness of the model.
- **Regression:** Linear Regression explains 92% of the variation in global sales (R2 = 0.92). The tree-based regressors score lower (R2 around 0.77 to 0.79) because a few very large hits are hard to predict with a small tree. Lasso is a little lower because it shrinks the slope.
- **Objectives:** all the objectives were achieved. The data was explored and tested with all SI Lab methods, multiple models were trained and compared, the best models were saved, and a working Streamlit GUI was built.

7. CONCLUSION

In this project we analysed the sales of 16,291 video games using all the SI Lab practicals, from web scraping and descriptive statistics to sampling, CLT, confidence intervals, hypothesis tests, ANOVA, chi-square and correlation. The analysis showed that game sales are highly skewed, North America is the biggest market, and sales depend on both genre and platform.

We trained 4 classification models to predict whether a game will be a hit and 5 regression models to predict its global sales, using North America sales as the input. The Decision Tree was selected as the best classifier with 95.8% accuracy and an F1-Score of 0.82, and Linear Regression was selected as the best regression model with R2 = 0.92. Both models are used in a Streamlit web app.

The main limitation is that the models use only North America sales, so they miss games that are popular mainly in Japan or Europe. The data also ends around 2016. In the future, the models can be improved by adding more features like EU and JP sales, genre, platform and publisher, by handling the class imbalance (for example with class weights or SMOTE), by tuning the models with GridSearchCV, and by deploying the app on Streamlit Community Cloud so that anyone can open it online.

8. GITHUB LINK

GitHub Repository URL: https://github.com/hixyt/video-game-sales-hit-prediction

The repository contains:

- B3_B1_07_SIPROJECT.ipynb: Colab notebook with all practicals and models.
- app.py: Streamlit GUI application code.
- best_classifier.pkl, scaler_classifier.pkl, best_model.pkl, scaler.pkl: trained model files.
- requirements.txt: Python libraries needed to run the project.
- README.md: project description and dataset link.

9. REFERENCES

[1] Video Game Sales dataset (vgsales.csv), Kaggle. https://www.kaggle.com/datasets/dandanjia/vgsales-csv

[2] VGChartz, original source of the sales data. https://www.vgchartz.com

[3] List of best-selling video games, Wikipedia (used for web scraping). https://en.wikipedia.org/wiki/List_of_best-selling_video_games

[4] Scikit-learn documentation. https://scikit-learn.org/stable/

[5] Pandas documentation. https://pandas.pydata.org/docs/

[6] SciPy statistical functions (scipy.stats) documentation. https://docs.scipy.org/doc/scipy/reference/stats.html

[7] Statsmodels documentation. https://www.statsmodels.org/stable/

[8] Streamlit documentation. https://docs.streamlit.io

[9] Matplotlib and Seaborn documentation. https://matplotlib.org and https://seaborn.pydata.org

[10] Beautiful Soup documentation. https://www.crummy.com/software/BeautifulSoup/bs4/doc/

[11] SI Lab practicals PR1 to PR7, Department of CSE (AIDS), Ramdeobaba University, Nagpur, 2026.